1. (2 Punkte) Sortiere die Zahlenfolge mit SelectionSort. Schreibe für die ersten drei Durchgänge je eine Zeile.

   `13 4 92 42 11 7 12`

Lösung:

```
4 13 92 42 11 7 12
4 7 92 42 11 13 12
4 7 11 42 92 13 12
```

2. (4 Punkte) Notiere den Code, der für den SelectionSort-Algorithmus fehlt. Gib auch die Stufe der Einrückung an.

```
def selection_sort(a):                  #E0
                                      ???(1)
        best = i                        #E2
        best_val = a[i]                 #E2
        for j in range(i+1,len(a)): #E2
                                      ???(2)
                best= j                 #E4
                best_val = a[j]         #E4
        a[best],a[i] = a[i],a[best]    #E2
```

Lösung:

```
(1) for i in range(len(a)-1):     #E1
(2) if a[j] < best_val:           #E3
```

3. (3 Punkte) In unserer Implementierung des SelectionSort-Algorithmus wurde ein print-Statement eingefügt. Welche Zahlenfolge wird bei dem angegebenen Aufruf von SelectionSort ausgegeben? (Der print-Parameter end=' ' sorgt dafür, dass die Zahlen nicht in einer neuen Zeile, sondern mit blank getrennt ausgegeben werden.)

```
def selection_sort(a):
    for i in range(len(a)-1):
        best = i
        best_val = a[i]
        for j in range(i+1,len(a)):
            if a[j] < best_val:
                best = j
                print(best,end= ' ')
                best_val = a[j]
        a[best],a[i] = a[i],a[best]

selection_sort([7,3,1,8,5,12,5,4])
```

Lösung:

```
1 2 4 7 4 6 6 7
```

4. (2 Punkte) Unsere Implementierung des SelectionSort-Algorithmus wurde mit anderen Variablennamen neu aufgeschrieben. Dabei haben sich Fehler eingeschlichen. Korrigiere die fehlerhaften Stellen.

```
def selection_sort(hugo):
    for kai in range(len(hugo)):
        otto = kai
        lena = hugo[kai]
        for j in range(kai+1,len(hugo)):
            otto = j
            lena = hugo[j]
        hugo[otto],hugo[kai] = hugo[kai],hugo[otto]
```

Lösung:

```
In Zeile 2: for kai in range(len(hugo)-1):
Nach Zeile 5 fehlt eine Zeile:
if hugo[j] < lena:
die beiden nächsten Zeilen müssen eingerückt werden,
so dass sie zu diesem if gehören.
```

5. (2 Punkte) Sortiere die Zahlenfolge mit BubbleSort. Schreibe für die ersten drei Durchgänge je eine Zeile.

```
2 22 14 25 1 13 9
```

Lösung:

```
2 14 22 1 13 9 25
2 14 1 13 9 22 25
2 1 13 9 14 22 25
```

6. (3 Punkte) In unserer Implementierung des Bubblesort-Algorithmus wurde ein print-Statement eingefügt. Welche Zahlenfolge wird bei dem angegebenen Aufruf von Bubblesort ausgegeben? (Der print-Parameter end=' ' sorgt dafür, dass die Zahlen nicht in einer neuen Zeile, sondern mit blank getrennt ausgegeben werden.)

```
def bubble_sort(a):
    getauscht = True
    while getauscht:
        getauscht = False
        for i in range(len(a)-1):
            if a[i] > a[i+1]:
                a[i],a[i+1]=a[i+1],a[i]
                print(a[i],end=' ')
                getauscht = True

bubble_sort([7,4,2,3,1])
```

Lösung:

```
4 2 3 1 2 3 1 1 1
```

7. (2 Punkte) In unserer Implementierung des Bubblesort-Algorithmus wurden Variablennamen geändert. Dabei haben sich Fehler eingeschlichen. Gib die Zeilennummern mit den Fehlern an und korrigiere die fehlerhaften Stellen. (Wir zählen Zeilennummern mit 1 beginnend).

```
def bubble_sort(alpha):
    beta = True
    while beta:
        beta = False
        for delta in range(len(alpha)-1):
            if alpha[gamma] > alpha[gamma+1]:
                alpha[gamma], alpha[gamma+1]=alpha[gamma+1], alpha[gamma]
            beta = True
```

Lösung:

```
In Zeile 5: for gamma in range(len(alpha)-1):
In Zeile 8: beta = True  muss weiter eingerückt werden (wie Zeile 7)
```

8. (2 Punkte) Sortiere die Zahlenfolge mit InsertionSort. Schreibe für die ersten drei Durchgänge je eine Zeile.

`12 5 9 3 15 7 1`

Lösung:

```
5 12 9 3 15 7 1
5 9 12 3 15 7 1
3 5 9 12 15 7 1
```

9. (5 Punkte) Die Liste `a = [24, 4, 17, 88, 42, 12, 7]` wird mit dem rekursiven mergeSort-Algorithmus aus dem Unterricht sortiert.
   a. Wieviel mal wird `merge` aufgerufen?
   b. Wieviel mal wird `mergeSort` aufgerufen? (der erste Aufruf zählt mit).

Lösung: a. 6 b. 13

Bei jedem Aufruf von `merge` wird eine Liste zurückgegeben. Notiere die Listen in der Reihenfolge, in der sie zurückgegeben werden.

Lösung:

```
4 17
4 17 24
42 88
7 12
7 12 42 88
4 7 12 17 24 42 88
```

10. (5 Punkte) Die Liste `22 41 43 7 42 19` wird mit quickSort sortiert.
Schreibe die ersten drei Protokollzeilen.

Lösung:

```
22 41 43 7 42 19 unten=0 oben=5 pivot=43
22 41 19 7 42 43 unten=0 oben=4 pivot=19
7 19 41 22 42 43 unten=0 oben=1 pivot=7
```

11. (2 Punkte) Quicksort erhält die Liste zur Sortierung. Schreibe die Protokollzeilen vor und nach dem ersten Durchgang.

`15 26 22 18 16 28 9 38 8`

Lösung:

```
15 26 22 18 16 28 9 38 8 unten=0 oben=8 pivot=16
15 8 9 16 18 28 22 38 26 unten=0 oben=3 pivot=8
```

12. (2 Punkte) Der nächste Quicksort-Durchgang bearbeitet die Liste von 0-3.

`15 8 9 16 18 28 22 38 26`

Schreibe die Protokollzeilen vor und nach dem Durchgang.

Lösung:

```
15 8 9 16 18 28 22 38 26 unten=0 oben=3 pivot=8
8 15 9 16 18 28 22 38 26 unten=1 oben=3 pivot=9
```

13. (2 Punkte) Der nächste Quicksort-Durchgang sortiert die Liste von 0-2. Der nächste wartende Abschnitt ist von 3-7.

`8 8 4 88 13 9 38 15`

Schreibe die Protokollzeilen vor und nach dem Durchgang.

Lösung:

```
8 8 4 88 13 9 38 15 unten=0 oben=2 pivot=8
4 8 8 88 13 9 38 15 unten=3 oben=7 pivot=9
```

14. (2 Punkte) Welche Komplexität hat die Laufzeit von BubbleSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf?

Lösung:

best: $O(n)$, average: $O(n^2)$, worst: $O(n^2)$, Platz: $O(1)$

15. (2 Punkte) Welche Komplexität hat die Laufzeit von InsertionSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf?

Lösung:

best: $O(n)$, average: $O(n^2)$, worst: $O(n^2)$, Platz: $O(1)$

16. (2 Punkte) Welche Komplexität hat die Laufzeit von MergeSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf?

Lösung:

best: $O(n \cdot \log n)$, average: $O(n \cdot \log n)$, worst: $O(n \cdot \log n)$, Platz: $O(n)$

17. (3 Punkte) (a) (1 Punkt) Eine Liste ist schon fast sortiert, nur wenige Elemente stehen ein oder zwei Plätze zu weit hinten. Welcher der Algorithmen SelectionSort, BubbleSort und InsertionSort ist dafür am besten geeignet? Begründe.

(b) (1 Punkt) Warum ist MergeSort im worst case schneller als QuickSort?

(c) (1 Punkt) Warum braucht MergeSort $O(n)$ zusätzlichen Speicherplatz, QuickSort aber nur $O(\log n)$?

Lösung:

a) InsertionSort: Jedes Element muss nur wenige Plätze nach vorne wandern, die Laufzeit ist fast $O(n)$. (BubbleSort braucht hier nur wenige Durchgänge und ist ebenfalls schnell.) SelectionSort braucht immer $O(n^2)$.

b) MergeSort teilt die Liste immer in zwei gleich große Hälften, es gibt also immer nur $\log n$ Ebenen. Bei QuickSort können bei ungünstigem Pivot-Element n Ebenen entstehen, dann ist die Laufzeit $O(n^2)$.

c) `merge` erzeugt neue Listen für die Ergebnisse. QuickSort tauscht nur innerhalb der Liste, Platz wird nur für die Variablen der $\log n$ Rekursionsebenen gebraucht.